

实验五

年级：_____2022_____专业：_____计算机科学与技术_____

姓名：_____姚知言_____学号：_____2211290_____

1 实验内容

实现对于二叉搜索树的如下操作：

1. 给定序列，使用逐点插入法构建二叉搜索树并输出，使用非递归的方法按照升序序列输出上述二叉搜索树的关键字；

2. 考虑一个包含整数值的二叉树（结构如下所示）。现有一个整数 K，任务是寻找并打印出所有符合以下要求的路径节点的值：

a. 路径上节点的总得分与给定的 K 相等，每个节点的得分由关键字值加上其所处的层级数决定（根节点的层级为 1，往下每层增加 1）。

b. 路径的走向是单向的，仅从上级节点到下级节点；

c. 路径的起点和终点不固定，可以是树中任意节点，向下延伸到任意节点；

要求打印输入的二叉树结构，并输出符合上述要求的路径。

说明：

(1) 第一问输入无重复的关键字，输出树时将结点的左右孩子结点在括号内输出，若为空则输出#

(2) 第二问根据示例自行设计构建二叉树，其中空子树用 # 表示

2 设计思想

该程序的设计思想如下：

(1)：主函数位于 op1

构造 BST 和 BSTnode 类，链式存储二叉搜索树。

类内定义构造函数 BST 和析构函数 ~BST（结合 release 函数使用）。

类内定义 addLchild, addRchild, defineroot 函数，分别用于定义根结点和为已有结点增加左右子结点。

类内定义 visit 函数用于以标准形式打印结点。类内定义 levelorder 函数用于层序遍历二叉树。构造队列 MQ 和 MQnode 类，用于层序遍历结点的暂存。在层序调用 visit 函数，完成打印。

类内定义 insert 函数，即完成以逐点插入法构造二叉搜索树。主要思路如下：

当根结点不存在的时候，该插入的结点即为根结点。

当根结点存在的时候，通过逐层与结点比较大小，即插入数值比结点数值大，就再和其右子结点比较，反之和其左子结点比较。（由于题目已经明确不存在重复的数值）当比较到外部结点时候，将该数值构成的结点插入。

类内定义 upper 函数，完成非递归的升序输出，主要思路如下：（当树为空时，直接 return）

若根结点无左子结点，即根结点为此时数值最小的结点，选择根结点的右子结点作为新的根结点，将根结点的数值打印并删除。

否则，最左查找直到找到第一个没有左子结点的结点，该结点为当前数值最小的结点（记作 A），其父结点记作 B，将 B 的左子结点指向 A 的右子结点，并将 A 的数值打印并删除。

(2)：主函数位于 op2

a. 数据的读入

当输入数值为 '#' 时，将数组数值赋值为不会引发歧义的极大值 $nu=2e9$ 。

当输入数值为实际数值时，利用字符串转数值的思路完成。若读入 '-' 则最终结果 $\times (-1)$ ，对每一位 $\times 10 +$ 下一位完成读入，将数据读入 B 数组中。

此部分时间复杂度应为字符串的长度 $O(x)$ ；

b. 数据的处理与二叉树的打印

曾考虑过以线性表实现，但因为若输入数据为链状则时间复杂度巨大，于是改为以链表实现。

构建新的 BT 类和 BNode 类，加入成员 father 指针和 depth 存储深度，以便后面实现。构造函数析构函数，addLchild, addRchild, defineroot, visit, levelorder 思路与第一问类似。

先定义第一个数值为根节点，并将根节点存入通过 MQ 实现的 BNode 队列中，当原数组没有遍历完的情况下，每次取出两个数值，若不为 nu，则其为 MQ.front 的左右子节点，通过 addLchild, addRchild 函数处理，并将新的结点放入队列，原 front 结点弹出，直到原数组完成遍历，结束操作。

层序遍历操作与第一问类似。

此部分时间复杂度可以为原来数组长度 $O(y)$ 。

c. 查找路径

主体思路:在上一步存储数据的时候预存层高和父结点指针元素，把每个结点作为子结点，从下向上遍历直至根节点，累加计算并通过数组暂存路线上的 data，即可遍历所有区间，并且较为优化。时间复杂度为 $O(n\log n)$ 。

在此过程中，若计算结果等于 k，满足题意输出条件，则将数组从后向前输出。

通过 getpath 完成查找和打印的操作。

3 程序效果

程序的运行效果，输入及输出的相关要求和具体执行结果如下所示：

3.1 输入

(1) in: 7 2 9 1 5 4 6

(2) in: [8, 3, 9, 11, 2, #, 13, 6, 7, #, 4, 8, #, #, 5, -2, #, 1, #, -3, #], K=28

3.2 输出

(1) out: 7(2, 9)

2(1, 5) 9(#, #)

1(#, #) 5(4, 6)

```

4( #, #) 6( #, #)
1 2 4 5 6 7 9
(2) out: 8(3, 9)
3(11, 2) 9( #, 13)
11(6, 7) 2( #, 4) 13(8, #)
6( #, 5) 7(-2, #) 4(1, #) 8(-3, #)
5( #, #) -2( #, #) 1( #, #) -3( #, #)
8 3 11
11 7 -2
13 8

```

4 核心代码

```

#include <iostream>
using namespace std;
const int nu=2e9;
template<class T>
class MQ;
template<class T>
class MQnode{
public:
    friend class MQ<T>;
private:
    T data;
    MQnode<T>* next;
};
template<class T>
class MQ{
public:
    MQ() {first=0;}
    ~MQ();
    bool empty() {if(!first)return true;else return false;}
    T front();
    void push(T x);
    void pop();
private:
    MQnode<T>* first;
};
template<class T>
MQ<T>::~~MQ() {

```

```

        auto* temp=first;
        while(first){
            first=temp->next;
            delete[] temp;
            temp=first;
        }
    }

    template<class T>
    void MQ<T>::push(T x) {
        auto* p=new MQnode<T>;
        p->data=x;p->next=0;
        if(empty()) {first=p;return;}
        auto* q=first;
        while(q->next)q=q->next;
        q->next=p;
    }

    template<class T>
    void MQ<T>::pop() {
        MQnode<T>*p=first->next;
        delete[] first;
        first=p;
    }

    template<class T>
    T MQ<T>::front() {
        if(empty())throw invalid_argument("No item!");
        return first->data;
    }

    template<class T>
    class BST;
    template<class T>
    class BSTnode{
    public:
        friend class BST<T>;
    private:
        T data;
        BSTnode<T> *Lchild,*Rchild;
    };

    template<class T>
    class BST{
    public:
        BST() {root=0;}
        ~BST();
        void release(BSTnode<T>* node);
        void insert(T x);

```

```

    void addLchild(BSTnode<T>* father, T x);
    void addRchild(BSTnode<T>* father, T x);
    void defineroot(T x);
    void visit(BSTnode<T>* node);
    void levelorder();
    void upper();
private:
    BSTnode<T>* root;
};

template<class T>
void BST<T>::release(BSTnode<T>* node) {
    if(node->Lchild) release(node->Lchild);
    if(node->Rchild) release(node->Rchild);
    delete[] node;
}

template<class T>
BST<T>::~~BST() {if(root) release(root);}

template<class T>
void BST<T>::addLchild(BSTnode<T>* father, T x) {
    if(father->Lchild!=0) throw invalid_argument("No place!");
    auto* temp=new BSTnode<T>;
    temp->data=x;
    temp->Lchild=temp->Rchild=0;
    father->Lchild=temp;
}

template<class T>
void BST<T>::addRchild(BSTnode<T>* father, T x) {
    if(father->Rchild!=0) throw invalid_argument("No place!");
    auto* temp=new BSTnode<T>;
    temp->data=x;
    temp->Lchild=temp->Rchild=0;
    father->Rchild=temp;
}

template<class T>
void BST<T>::defineroot(T x) {
    if(root) throw invalid_argument("No place!");
    root=new BSTnode<T>;
    root->data=x;
    root->Lchild=root->Rchild=0;
}

template<class T>
void BST<T>::insert(T x) {
    if(!root) {defineroot(x);return;}
    BSTnode<T>* temp=root;

```

```

while(true) {
    if(temp->data<x) {
        if(temp->Rchild) temp=temp->Rchild;
        else {addRchild(temp, x); break;}
    }
    else {
        if(temp->Lchild) temp=temp->Lchild;
        else {addLchild(temp, x); break;}
    }
}
}

template<class T>
void BST<T>::visit(BSTnode<T>* node) {
    cout<<node->data<<' (';
    if(node->Lchild) cout<<node->Lchild->data;
    else cout<<' #' ;
    cout<<',';
    if(node->Rchild) cout<<node->Rchild->data;
    else cout<<' #' ;
    cout<<") ";
}

template<class T>
void BST<T>::levelorder() {
    MQ<BSTnode<T>*> order;
    int count=0, countc=1, countt=0;
    if(root!=0)
        order.push(root);
    while(!order.empty()) {
        visit(order.front()); count++;
        if(order.front()->Lchild) order.push(order.front()->Lchild);
        else countt++;
        if(order.front()->Rchild) order.push(order.front()->Rchild);
        else countt++;
        if(count==countc) {cout<<' \n' ; countc=countc*2+1; count+=countt; countt*=2;}
        order.pop();
    }
}

template<class T>
void BST<T>::upper() {
    while(root) {
        BSTnode<T> *temp = root;
        if (!temp->Lchild) {
            cout << temp->data << ' ';
            root=temp->Rchild;
        }
    }
}

```

```

        delete[] temp;
    }
    else{
        while(temp->Lchild->Lchild)
            temp=temp->Lchild;
        cout<<temp->Lchild->data<<' ';
        BSTnode<T>*temp1=temp->Lchild;
        temp->Lchild=temp->Lchild->Rchild;
        delete[] temp1;
    }
}
cout<<'\\n';
}

template<class T>
class BT;
template<class T>
class BTnode{
public:
    friend class BT<T>;
private:
    T data;
    BTnode<T> *Lchild,*Rchild,*Father;
    int depth;
};

template<class T>
class BT{
public:
    BT() {root=0;}
    ~BT();
    void release(BTnode<T>* node);
    void addLchild(BTnode<T>* father,T x);
    void addRchild(BTnode<T>* father,T x);
    void defineroot(T x);
    void make(int*bt,int& length);
    void visit(BTnode<T>* node);
    void levelorder();
    void getpath(BTnode<T>* m,int k);
    BTnode<T>* getroot();
private:
    BTnode<T>* root;
};

template<class T>
void BT<T>::release(BTnode<T>* node) {
    if(node->Lchild)release(node->Lchild);

```

```

        if(node->Rchild)release(node->Rchild);
        delete[]node;
    }
    template<class T>
    BT<T>::~~BT() {if(root) release(root);}
    template<class T>
    void BT<T>::addLchild(BTnode<T>* father,T x) {
        if(father->Lchild!=0)throw invalid_argument("No place!");
        auto* temp=new BTnode<T>;
        temp->data=x;
        temp->Lchild=temp->Rchild=0;
        temp->Father=father;
        temp->depth=father->depth+1;
        father->Lchild=temp;
    }
    template<class T>
    void BT<T>::addRchild(BTnode<T>* father,T x) {
        if(father->Rchild!=0)throw invalid_argument("No place!");
        auto* temp=new BTnode<T>;
        temp->data=x;
        temp->Lchild=temp->Rchild=0;
        temp->Father=father;
        temp->depth=father->depth+1;
        father->Rchild=temp;
    }
    template<class T>
    void BT<T>::defineroot(T x) {
        if(root)throw invalid_argument("No place!");
        root=new BTnode<T>;
        root->data=x;
        root->Lchild=root->Rchild=0;
        root->Father=0;
        root->depth=1;
    }
    template<class T>
    BTnode<T>* BT<T>::getroot() {
        if(!root)throw invalid_argument("Empty!");
        return root;
    }
    template<class T>
    void BT<T>::make(int*bt,int& length) {
        defineroot(bt[1]);
        MQ<BTnode<T>*> mk;
        mk.push(root);
    }

```

```

    int fb=1;
    while(fb<length){
        if(bt[fb]!=nu){
            addLchild(mk.front(),bt[fb]);
            mk.push(mk.front()->Lchild);
        }
        if(bt[fb]!=nu){
            addRchild(mk.front(),bt[fb]);
            mk.push(mk.front()->Rchild);
        }
        mk.pop();
    }
}

template<class T>
void BT<T>::visit(BTnode<T>* node){
    cout<<node->data<<' (';
    if(node->Lchild)cout<<node->Lchild->data;
    else cout<<' #' ;
    cout<<',';
    if(node->Rchild)cout<<node->Rchild->data;
    else cout<<' #' ;
    cout<<" ) ";
}

template<class T>
void BT<T>::levelorder(){
    MQ<BTnode<T>*> order;
    int count=0,countc=1,countt=0;
    if(root!=0)
        order.push(root);
    while(!order.empty()){
        visit(order.front());count++;
        if(order.front()->Lchild)order.push(order.front()->Lchild);
        else countt++;
        if(order.front()->Rchild)order.push(order.front()->Rchild);
        else countt++;
        if(count==countc){cout<<' \n';countc=countc*2+1;count+=countt;countt*=2;}
        order.pop();
    }
}

template<class T>
void BT<T>::getpath(BTnode<T>* m,int k){
    BTnode<T>* ancestor=0;
    int* dt=new int[m->depth];
    int score=0,position=0;

```

```

do{
    if(!ancestor)ancestor=m;
    else ancestor=ancestor->Father;
    score+=ancestor->data+ancestor->depth;
    dt[position++]=ancestor->data;
    if(score==k) {
        for(int g=position-1;g>=0;g--) {
            cout<<dt[g]<<' ';
        }
        cout<<' \n' ;
    }
}
while(ancestor!=root);
if(m->Lchild)getpath(m->Lchild,k);
if(m->Rchild)getpath(m->Rchild,k);
}

void op1() {
    BST<double> A;
    double a;
    do{cin>>a;A.insert(a);}
    while(getchar()!='\n');
    A.levelorder();
    A.upper();
}

void op2() {
    int B[100000], length=0, num=0, k;
    bool p=0;
    char u;
    cin>>u;
    while(u!=' '){
        cin>>u;
        if(u=='#') {
            B[++length]=nu;
            cin>>u;
        }
        else{
            while(u!=',') {
                if(u=='-')p=1;
                else {num*=10, num+=(u-'0');}
                cin>>u;
            }
            if(p) num*=-1;
            B[++length]=num; num=0, p=0;
        }
    }
}

```

```

    }
    while(getchar()!=' ');
    cin>>k;
    BT<int>tree;
    tree.make(B, length);
    tree.levelorder();
    tree.getpath(tree.getroot(), k);
}
int main() {
    op1();
    op2();
    return 0;
}

```

5 总结

本次实验，复习了二叉树的结构和组成方式，复习了二叉搜索树的构建和二叉搜索树用于排序的算法，复习了树的层序遍历输出的方法，复习了通过父子结点关系建树的方法。同时，利用（2）中的小问题更好的体会了二叉树的结构与连接方式。